

GUIA DE INSTALAÇÃO

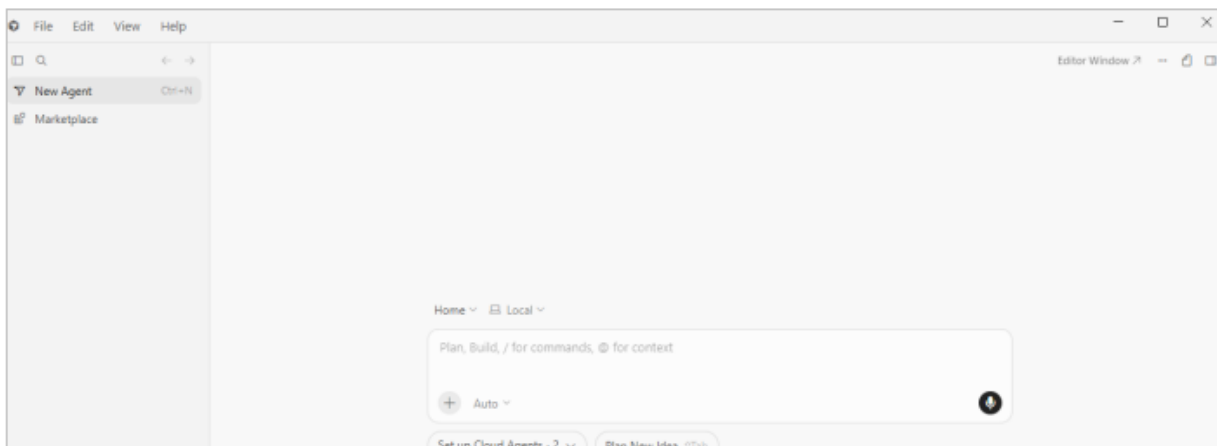
Setup do Ambiente no Windows

Cursor · Git · GitHub · Node.js · Python · Jupyter · MCPs

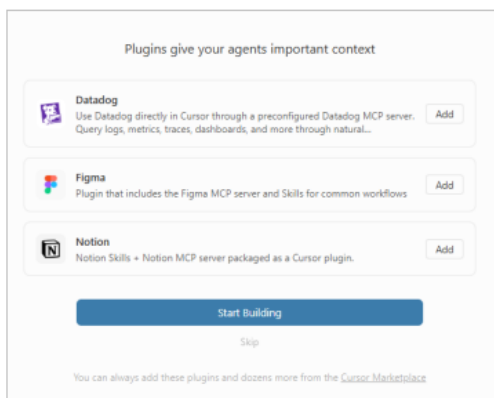
Este guia descreve, passo a passo, a instalação e configuração de todas as ferramentas necessárias para o ambiente de desenvolvimento no Windows. Siga os passos na ordem indicada.

Passo 1 Instalar o Cursor

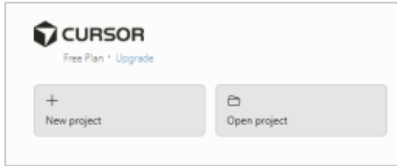
- ▶ Acesse <https://cursor.com>
- ▶ Clique em **Download for Windows** e instale.
- ▶ **Abra o Cursor e faça login** com sua conta. Crie uma se ainda não tiver (**use o email institucional do IPOG**).
- ▶ Após o login, a tela de chat será aberta. Troque para **Editor Windows** (opção no canto superior direito).



- ▶ Você pode escolher adicionar os 3 plugins e depois clicar em **Start Building**.



- ▶ Selecione **New Project**.



Passo 2 Instalar o Git

- ▶ Acesse <https://git-scm.com/install/windows>
- ▶ Faça o download e instale: latest **(2.54.0) x64** version of **Git for Windows**.
- ▶ Para testar, abra o **Terminal (cmd)** e digite:

```
git --version
```

- ▶ Deve aparecer algo como **git version 2.54.0.windows.1**.

Passo 3 Configurar conta no GitHub

- ▶ Crie uma conta no GitHub (**use o email institucional do IPOG**).
- ▶ Faça login no GitHub.
- ▶ Prepare a **autenticação por token**:
 1. Vá em **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)**.
 2. Clique em **Generate new token (classic)**.
 3. Dê um nome descritivo (ex.: *hackathon2026*).
 4. Marque o escopo **repo** (acesso completo a repositórios).
 5. Clique em **Generate token**.
 6. **COPIE O TOKEN** — depois de fechar a página, ele não será mais exibido.

⚠ Importante: guarde esse token em um lugar seguro (NÃO dentro do Git!). Você o usará como senha quando o git push pedir autenticação.

Passo 4 Configurar nome e e-mail no Git

Essencial para identificar quem fez cada mudança.

- ▶ Abra o **Terminal (cmd)** do Windows e digite:

```
git config --global user.name "Seu Nome Completo"
```

```
git config --global user.email "seu-email@institucional.br"
```

⚠ **Atenção:** use o mesmo e-mail que você cadastrou no GitHub, senão seus commits não vão aparecer associados à sua conta.

- ▶ Para salvar suas credenciais e não digitar o token toda hora:

```
git config --global credential.helper store
```

Passo 5 Instalar Node.js e npm

- ▶ Abra o **Terminal (cmd)** do Windows, baixe e instale o Chocolatey:

```
powershell -c "irm https://community.chocolatey.org/install.ps1 | iex"
```

- ▶ No Terminal (cmd), baixe e instale o Node.js:

```
choco install nodejs --version="24.15.0"
```

- ▶ Verifique a versão do Node.js:

```
node -v
```

Deve exibir algo como **v24.15.0**.

- ▶ Verifique a versão do npm:

```
npm -v
```

Deve imprimir algo como **11.12.1**.

Passo 6 Instalar Python 3

- ▶ Verifique se o Python já está instalado. No **Terminal (cmd)**, digite:

```
python3 --version  
ou  
python --version
```

Caso o Python não esteja instalado, siga os procedimentos descritos em:

<https://python.org.br/instalacao-windows/>

Passo 7 Instalação das Ferramentas

- ▶ No Terminal do Windows, crie uma pasta para o projeto e entre nela:

```
mkdir hackathon
cd hackathon
```

- ▶ Abra o Cursor e entre na pasta do projeto: hackathon.
- ▶ Abra o Terminal dentro do Cursor (**Terminal > New Terminal**).
- ▶ No Terminal dentro do Cursor, ative o venv na pasta atual:

```
.\venv\Scripts\Activate.ps1
```

- ▶ Quando funcionar, o terminal ficará parecido com:

```
(venv) PS C:\Users\joelma\hackathon>
```

- ▶ Se aparecer erro de política de execução, execute primeiro:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

- ▶ Depois, tente novamente:

```
.\venv\Scripts\Activate.ps1
```

Passo 8 Instalar Jupyter Lab e dependências

- ▶ No Terminal dentro do Cursor, instale o Jupyter Lab e ferramentas necessárias:

```
pip install jupyterlab==4.4.1 jupyter-collaboration==4.0.2 ipykernel
pip install datalayer_pycrdt==0.12.17
```

Passo 9 Iniciar o Jupyter Lab

- ▶ No Terminal dentro do Cursor, digite:

```
jupyter lab --port 8888 --IdentityProvider.token MY_TOKEN --ServerApp.root_dir ./
```

 **O que é esse MY_TOKEN?** É uma senha local que protege seu Jupyter Lab. Para o hackathon, pode usar algo simples como hackathon123.

⚠ Importante: o Jupyter Lab fica rodando enquanto esse terminal estiver aberto — não feche! Abra outro terminal para outras coisas.

Passo 10 Configurar o MCP Context7

- ▶ Abra o Terminal no próprio Cursor e ative (ou crie) o ambiente virtual:

```
.\env\Scripts\Activate.ps1
```

- ▶ Abra a pasta do projeto no Cursor e, no Terminal do Cursor, faça login no Context7 (vai abrir um navegador — logue com o email da sua conta no Cursor).

```
npx -y ctx7 login
```

Vai aparecer uma tela para fazer login ou criar a conta (signup) e depois dar permissão de acesso.

- ▶ Configure no Cursor:

```
npx ctx7 setup --cursor -y
```

- ▶ Reinicie o Cursor (feche completamente e abra novamente).
- ▶ Reative o ambiente virtual no Terminal dentro do Cursor:

```
.\env\Scripts\Activate.ps1
```

Para testar, no chat do Cursor digite:

```
Use o MCP Context7 para explicar boas práticas de APIs REST em Spring Boot
```

Se a resposta vier com referências reais à documentação atual, está funcionando.

Passo 11 MCP Jupyter — executar células de notebook

- ▶ Abra a pasta do projeto no Cursor, crie a pasta `.cursor` na raiz do seu projeto e, dentro dela, um arquivo `mcp.json`:

```
{
  "mcpServers": {
    "jupyter": {
      "command": "C:\\\\Users\\joelma\\hackathon\\env\\Scripts\\python.exe",
      "args": ["-m", "uv", "tool", "run", "jupyter-mcp-server@latest"],
```

```

    "env": {
      "JUPYTER_URL": "http://localhost:8888",
      "JUPYTER_TOKEN": "MY_TOKEN",
      "ALLOW_IMG_OUTPUT": "true"
    }
  }
}
}

```

 **Substitua** MY_TOKEN pelo token que você definiu ao iniciar o Jupyter Lab.

Passo 12 MCP Filesystem — IA acessar seu projeto

- ▶ No Terminal do Cursor, instale globalmente:

```
npm install -g @modelcontextprotocol/server-filesystem
```

- ▶ Adicione ao mcp.json (junte com a configuração do Jupyter):

```

{
  "mcpServers": {
    "jupyter": {
      "command": "C:\\\\Users\\joelma\\hackathon\\venv\\Scripts\\python.exe",
      "args": ["-m", "uv", "tool", "run", "jupyter-mcp-server@latest"],
      "env": {
        "JUPYTER_URL": "http://localhost:8888",
        "JUPYTER_TOKEN": "MY_TOKEN",
        "ALLOW_IMG_OUTPUT": "true"
      }
    },
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "C:\\\\Users\\joelma\\hackathon"
      ]
    }
  }
}

```

💡 **Substitua** o caminho pelo caminho completo da pasta do seu projeto. Exemplo:
C:\\Users\\joelma\\hackathon

Passo 13 Ativar os MCPs no Cursor

Depois de salvar o `mcp.json`:

- ▶ **Feche o Cursor completamente** (não basta fechar a janela — feche o programa todo).
- ▶ Abra o Cursor novamente.
- ▶ Reative o ambiente virtual: `.\venv\\Scripts\\Activate.ps1`
- ▶ Vá em **File → Preferences → Cursor Settings → Tools and MCP**.
- ▶ Marque os MCPs que você configurou (**filesystem, jupyter, context7**).
- ▶ **Reinicie o Cursor novamente**.
- ▶ Reative o ambiente virtual: `.\venv\\Scripts\\Activate.ps1`
- ▶ Inicie o Jupyter Lab:

```
jupyter lab --port 8888 --IdentityProvider.token MY_TOKEN --ServerApp.root_dir ./
```

Para testar o filesystem, no chat do Cursor:

Use o MCP filesystem para listar todos os arquivos do meu projeto

Para testar o Jupyter, com o Jupyter Lab rodando, no chat:

Abra o notebook `01_exploracao.ipynb`, crie uma célula que importa pandas e executa `print("Olá!")`. Me diga exatamente o que apareceu.

✓ Verificação final dos MCPs

Dentro do Cursor, em Settings → Tools and MCP, devem aparecer:

- ▶ ☒ **context7** (verde)
- ▶ ☒ **jupyter** (verde)
- ▶ ☒ **filesystem** (verde)